



Diseño de Software

Taller Sumativo

Grupo#5

Integrantes:

- NAVARRETE FIGUEROA JOSE ANTONIO
- SALAZAR GORDILLO ELKIN MATIAS
- ZOUEN VELEZ NEJEH YOUSSEF

Índice

Composing Methods:	2
Extract Method	2
Inline Temp.....	3
Simplifying Conditional Expressions:	3
Replace Nested Conditional with Guard Clauses.....	3
Replace Conditional with Polymorphism	4
Dealing with Generalization:	5
Pull Up Field	5
Push Down Field	5
Pull Up Constructor Body	6
Pull Up Method	6
Moving Features between Objects:	6
Move Method	6
Enlace del GITHUB:	7

Tecnicas de factorizacion

Composing Methods:

Extract Method

Clase y Metodo: Empleado.calcularSalario()

En este método mezcla al menos tres responsabilidades distintas en un solo bloque: validar que el salario y las horas trabajadas sean válidos, calcular el pago de horas extra, y calcular el bono según el departamento. Mantenerlo así hace que el método sea difícil de leer y de probar de forma aislada (no se puede testear "el cálculo de horas extra" sin ejecutar también la validación y el switch de departamento haciendo que sea pesado el metodo) y tambien que cualquier cambio futuro en una de esas reglas obliga a tocar un método que hace demasiadas cosas a la vez, aumentando el riesgo de introducir errores en partes que no se querían modificar.

Como una solucion que hemos tomado seria la extraccion de métodos como validarDatosSalario(), calcularPagoHorasExtra() y calcularBonoPorDepartamento(), cada uno queda nombrado según su intención, se puede probar por separado, y calcularSalario() termina siendo una secuencia legible de tres llamadas

```
public double calcularSalario() {
    validarDatos();
    return salarioBase + calcularPagoHorasExtra() + calcularBonificacionDepartamento();
}

private void validarDatos() {
    validarSalarioBase();
    validarHorasTrabajadas();
}

private void validarSalarioBase() {
    if (salarioBase <= 0) {
        throw new IllegalArgumentException(s: "El salario debe ser mayor o igual a 0");
    }
}

private void validarHorasTrabajadas() {
    if (horasTrabajadas < 0) {
        throw new IllegalArgumentException(s: "Las horas trabajadas deben ser mayor o igual a 0");
    }
}

private double calcularPagoHorasExtra() {
    if (horasTrabajadas > 40) {
        return (horasTrabajadas - 40) * 50; // Pago de horas extra
    }
    return 0;
}
```

Inline Temp

En la clase y específicamente en el metodo: EmpleadoPorHoras.calcularSalario()

En esta de aquí ocurre lo contrario al caso anterior que tenemos una variable temporal (salario) que se calcula y se usa una única vez, inmediatamente después, solo para retornarla. Mantener esa variable no aporta claridad ni reutilización, solo añade una línea extra que el lector debe rastrear mentalmente para confirmar que no se usa en ningún otro lado. Al aplicar Inline Temp, la expresión `super.getHorasTrabajadas() * super.getTarifaHora()` se retorna directamente, el método queda en una sola línea y se elimina ruido innecesario sin perder legibilidad

```
public double calcularSalario() {
    validarDatos();
    return salarioBase + calcularPagoHorasExtra() + calcularBonificacionDepartamento();
}
```

Simplifying Conditional Expressions:

Replace Nested Conditional with Guard Clauses

En la clase y metodo de Empleado.calcularSalario(), lo que hemos visto es que las validación actual anida un if dentro de otro (if salarioBase>0 { if horasTrabajadas>=0 {...} else throw } else throw) por lo cual obliga a leer varios niveles para entender cuál es

el camino del método y cuáles son los casos de error. Esto es exactamente el patrón que la técnica busca corregir: mientras más anidado esté el código, más cuesta seguir el flujo y más fácil es que alguien inserte lógica nueva en el nivel equivocado. Por lo que hemos visto en grupo es reemplazando por cláusulas de guarda (if (salarioBase <= 0) throw ...; if (horasTrabajadas < 0) throw ...; seguidas del cálculo normal sin anidamiento) el método queda plano, y las condiciones de error quedan claramente separadas del caso

Replace Conditional with Polymorphism

En la clase y método **Empleado.calcularSalario()**, exactamente switch. Ese switch que asigna un bono fijo según el nombre del departamento ("Sistemas" o "Contabilidad") es un caso típico de "type code" ya que cada vez que la empresa agregue un nuevo departamento con reglas de bono propias, alguien tendrá que volver a este método y añadir un nuevo case, con el riesgo de olvidar algún lugar donde se repita esta lógica. Convirtiendo el departamento en una jerarquía de tipos (por ejemplo, una clase Departamento con subclases o implementaciones que exponen calcularBono()), cada regla queda encapsulada en su propia clase y agregar un departamento nuevo no implica modificar el código existente, solo agregar una clase nueva (cumpliendo además con el principio abierto/cerrado)

```
public class DepartamentoFactory {  
    public static Departamento crear(String nombre) {  
        if (nombre == null) {  
            return new DepartamentoGenerico(nombre: null);  
        }  
        switch (nombre) {  
            case "Sistemas":  
                return new DepartamentoSistemas();  
            case "Contabilidad":  
                return new DepartamentoContabilidad();  
            default:  
                return new DepartamentoGenerico(nombre);  
        }  
    }  
}
```

```

public class DepartamentoGenerico implements Departamento {
    private final String nombre;

    public DepartamentoGenerico(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public String getNombre() {
        return nombre;
    }

    @Override
    public double calcularBono() {
        return 0;
    }
}

```

Dealing with Generalization:

Pull Up Field

El campo genero estaba duplicado y declarado en tres subclases diferentes: EmpleadoFijo, EmpleadoPorHoras, EmpleadoTemporario lo que generaba redundancia en el código.

Solución:

Se movió el atributo genero a la clase padre Empleado, junto con sus métodos getter y setter.

```

public class Empleado {
    private String nombre;
    private double salarioBase;
    private int horasTrabajadas;
    private Departamento departamento;
    private double tarifaHora;
    private String genero;
}

```

```

public String getGenero() {
    return genero;
}

public void setGenero(String genero) {
    this.genero = genero;
}

```

Push Down Field

Existían atributos en la clase padre Empleado que no eran utilizados por todas las subclases. Esto hacía que algunas clases heredaran información que realmente no necesitaban.

Solución:

Los atributos que solo pertenecen a un tipo específico de empleado se movieron desde la clase Empleado hacia la subclase correspondiente. De esta manera, cada clase contiene únicamente los datos que realmente utiliza.

Pull Up Constructor Body

Cada subclase inicializaba manualmente los atributos heredados, repitiendo el mismo código.

Solución:

Se creó un constructor completo en la clase Empleado y las subclases utilizan super() para inicializar los atributos heredados

```
public class EmpleadoFijo extends Empleado {
    private double bonoAnual;

    public EmpleadoFijo(String nombre, double salarioBase, int horasTrabajadas, String departamento, double
        super(nombre, salarioBase, horasTrabajadas, tarifaHora: 0, departamento, genero);
        this.bonoAnual = bonoAnual;
    }

    public EmpleadoPorHoras(String nombre, int horasTrabajadas, double tarifaHora, String departamento, Str
        super(nombre, salarioBase: 0, horasTrabajadas, tarifaHora, departamento, genero);
        super.setSalarioBase(calcularSalario());
    }
```

Pull Up Method

El método imprimirDetalles() estaba implementado en varias subclases con código muy similar e incluso algunos errores.

Solución:

Se implementó el método en la clase Empleado. Para que las subclases lo utilicen

```
private void imprimirDetallesBasicos() {
    System.out.println("Nombre: " + nombre);
    System.out.println("Genero: " + genero);
    System.out.println("Salario: " + salarioBase);
    System.out.println("Horas trabajadas: " + horasTrabajadas);
    System.out.println("Departamento: " + departamento.getNombre());
}
```

Esto en clase empleado

Moving Features between Objects:

Move Method

En la clase y metodo de **BuscadorEmpleado.buscarEmpleadoPorNombre()**

En este método no usa ningún estado propio de **BuscadorEmpleado** (de hecho es static y no tiene atributos); toda la información que necesita la lista de empleados ya vive en **Empresa**, que además ya tiene un método análogo (**obtenerEmpleadosPorHorasTrabajadasaPartirDe**) para consultar empleados. Tener esta lógica en una clase aparte fuerza a quien la usa a pasar la lista como parámetro cada vez, en vez de simplemente preguntarle a la empresa "dame el empleado que se llama X", y dispersa la responsabilidad de "consultar empleados" en dos clases distintas. Moviendo el método a **Empresa** como **buscarEmpleadoPorNombre(String nombre)** (usando el campo **empleados** interno), la responsabilidad de búsqueda queda centralizada junto con el resto de las operaciones sobre la lista de empleados que esta en la clase de empresa

```
public class Empresa {  
    public List<Empleado> obtenerEmpleadosPorHorasTrabajadasaPartirDe(int horas) {  
        List<Empleado> empleadosPorTipo = new ArrayList<>();  
        for (Empleado empleado : empleados) {  
            agregarEmpleadoSiTieneHorasSuficientes(empleadosPorTipo, empleado, horas);  
        }  
        return empleadosPorTipo;  
    }  
}
```

Enlace del GITHUB:

<https://github.com/nezouein/taller12-refactoring/tree/main>

